University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 2

Date: February 24, 2026

Group Number: 4

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Tim Jung	76511353	m8n5t@ugrad.cs.ubc.ca	timjung1224@gmail.com
Teagan Richman-Taylor	69498327	w7p8z@ugrad.cs.ubc.ca	teaganrt@student.ubc.ca
William He	15552508	p9u9y@ugrad.cs.ubc.ca	williamhe937@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

CPSC304 Project Milestone 2

1 Description

The domain of our application is art collection management. Our application will serve art dealerships/management companies who are responsible for large private art collections. Our users are employed by clients with valuable works of art that may be loaned to museums. Each piece of art may have specific security and maintenance requirements. The art dealership must also maintain a record of each art work's current museum and the contact information for the conservator. Our application will offer a convenient way for art dealerships to store and access all of this information.

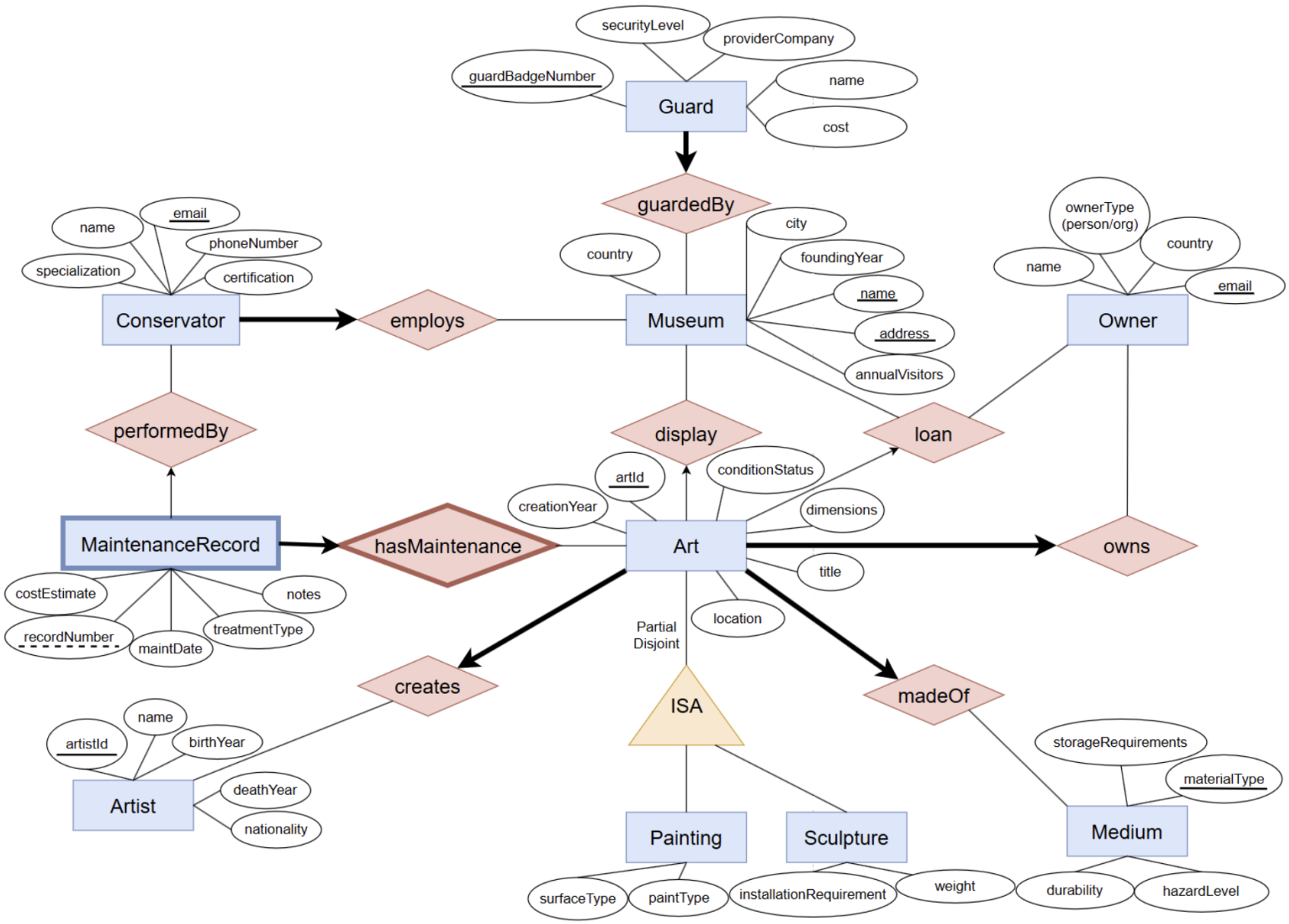
The applications database will model the most important information for these collection managers. We will represent each client and all of their art. Each piece of artwork will be related to its medium, artist and the current museum. We will also represent the required security and maintenance for a piece of art and the conservator who is responsible for said maintenance. For example, a client with a large art collection passes away and leaves all his art to their family. The art dealer who managed this client's collection may then receive a request from the next of kin for a dossier summarizing the client's portfolio. The dealer can use this database to quickly obtain a comprehensive list of the client's art (including its artist and medium), where it is housed, who the conservator is and how to contact them, as well as the security and maintenance requirements. The client's family can use all of this information to decide if they will sell the artwork or continue to maintain the collection.

2 Technology

For this project, our team will use JavaScript and Oracle database.

3 ER Diagram

We define that art has one medium for simplicity. Rationale for not including TA suggestion: Our TA suggested modifying our ER diagram so that Art can be madeOf multiple Medium. We decided to not include this change because we expect the medium entities to be quite broad (e.g. watercolour, oil on canvas, clay) and most art should be composed of only one medium.


securityLevel
providerCompany
guardBadgeNumber
Guard
name
cost
guardedBy
name
email
phoneNumber
specialization
certification
Conservator
employs
country
Museum
city
foundingYear
name
address
annualVisitors
ownerType (person/org)
country
name
email
Owner
performedBy
display
loan
artId
conditionStatus
creationYear
dimensions
MaintenanceRecord
hasMaintenance
Art
owns
title
costEstimate
notes
recordNumber
maintDate
treatmentType
Partial Disjoint
location
creates
ISA
madeOf
name
artistId
birthYear
Artist
deathYear
nationality
storageRequirements
materialType
Painting
Sculpture
Medium
surfaceType
paintType
installationRequirement
weight
durability
hazardLevel

4 Schema

Note:

- display,
- guardedBy,
- employs,
- performedBy,
- hasMaintenance,
- owns,
- madeOf,
- creates

have been merged into other tables

SCHEMA Art(artId: int, museumName: char[20], museumAddress: char[20], ownerEmail: char[20], creationYear: int, conditionStatus: char[20], dimensions: char[20], title: char[20], materialType: char[20], artistId: int, location: char[20])
KEYS artId is the primary key
artId is a key
(museumName, museumAddress) is a foreign key referencing Museum.museumAddress
ownerEmail is a foreign key referencing Owner.ownerEmail
artistId is a foreign key referencing Artist.artistId
materialType is a foreign key referencing Medium.materialType
CONSTRAINTS ownerEmail is not null
artistId is not null
materialType is not null
USER DEFINED CONSTRAINTS title is not null

SCHEMA Sculpture(artId: int, installationRequirement: char[20], weight: char[20])
KEYS artId is the primary key
artId is a key
artId is a foreign key referencing Art.artId
CONSTRAINTS None
USER DEFINED CONSTRAINTS None

SCHEMA Painting(artId: int, surfaceType:char[20], paintType: char[20])
KEYS artId is the primary key
artId is a key
artId is a foreign key referencing Art.artId
CONSTRAINTS None
USER DEFINED CONSTRAINTS None

SCHEMA Owner(ownerEmail: char[20], ownerName: char[20], ownerCountry: char[20], ownerType: char[20])
KEYS ownerEmail is the primary key
ownerEmail is a key
CONSTRAINTS None
USER DEFINED CONSTRAINTS None

SCHEMA Artist(artistId: char[20], artistName: char[20], birthYear: int, deathYear: int, nationality: char[20])
KEYS artistId is the primary key
artistId is a key
CONSTRAINTS None
USER DEFINED CONSTRAINTS artistName is not null

SCHEMA Museum(museumName: char[20], museumAddress: char[20], city: char[20], foundingYear: int, annualVisitors: int, country: char[20])
KEYS museumName, museumAddress is the primary key
museumName, museumAddress is a key
CONSTRAINTS None
USER DEFINED CONSTRAINTS city is not null

SCHEMA Guard(guardBadgeNumber: int, museumName: char[20], museumAddress: char[20], securityLevel: int, providerCompany: char[20], name: char[20], cost: int)
KEYS guardBadgeNumber is the primary key
guardBadgeNumber is a key
museumName is a foreign key referencing Museum.museumName
museumAddress is a foreign key referencing Museum.museumAddress
CONSTRAINTS museumName is not null
museumAddress is not null
USER DEFINED CONSTRAINTS name is not null
providerCompany is not null

SCHEMA Conservator(conservatorEmail: char[20], conservatorName: char[20], phoneNumber: char[20], specialization: char[20], certification: char[20], museumName: char[20], museumAddress: char[20])
KEYS conservatorEmail is the primary key
conservatorEmail is a key
museumName is a foreign key referencing Museum.museumName
museumAddress is a foreign key referencing Museum.museumAddress
CONSTRAINTS museumName is not null
museumAddress is not null
USER DEFINED CONSTRAINTS coservatorName is not null
phoneNumber is not null

SCHEMA Medium(materialType: char[20], storageRequirements: char[20], hazardLevel: char[20], durability: char[20])
KEYS materialType is the primary key
materialType is a key
CONSTRAINTS None
USER DEFINED CONSTRAINTS None

SCHEMA MaintenanceRecord(artId: int, recordNumber: int, costEstimate: decimal(10,2), maintDate: date, treatmentType: char[20], notes: char[20], conservatorEmail: char[20])
KEYS artId, recordNumber is the primary key
artId, recordNumber is a key
artId is a foreign key referencing Art.artId
conservatorEmail is a foreign key referencing Conservator.conservatorEmail
CONSTRAINTS None
USER DEFINED CONSTRAINTS costEstimate is not null maintDate is not null

---

SCHEMA Loan(artId: int, museumName: char[20], museumAddress: char[20], ownerEmail: char[20])
KEYS artId, museumName, museumAddress, ownerEmail is the primary key
artId, museumName, museumAddress, ownerEmail is a key
artId is a foreign key referencing Art.artId
museumName, museumAddress is a foreign key referencing Museum.museumName, Museum.museumAddress
ownerEmail is a foreign key referencing Owner.ownerEmail
CONSTRAINTS artId is unique
USER DEFINED CONSTRAINTS None

5 FD

Art(artId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType, artistId, location)
artId -> artistId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType
museumAddress -> location

Sculpture(artId, installationRequirement, weight)
artId -> installationRequirement, weight

Painting(artId, surfaceType, paintType)
artId -> surfaceType, paintType

Owner(ownerEmail, ownerName, ownerCountry, ownerType)

ownerEmail -> ownerName, ownerCountry, ownerType

Artist(artistId, artistName, birthYear, deathYear, nationality)

artistId -> artistName, birthYear, deathYear, nationality

Museum(museumName, museumAddress, city, foundingYear, annualVisitors, country)

museumName, museumAddress -> city, foundingYear, annualVisitors, country

museumAddress -> city, country

Guard(guardBadgeNumber, museumName, museumAddress, securityLevel, providerCompany, name, cost)

guardBadgeNumber -> museumName, museumAddress, securityLevel, providerCompany, name

securityLevel, providerCompany -> cost

Conservator(conservatorEmail, conservatorName, phoneNumber, specialization, certification, museumName, museumAddress)

conservatorEmail -> museumName, museumAddress, conservatorName, phoneNumber, specialization, certification

certification -> specialization

Medium(materialType, storageRequirements, hazardLevel, durability)

materialType -> storageRequirements, hazardLevel, durability

hazardLevel -> durability

durability -> storageRequirements

MaintenanceRecord(artId, recordNumber, costEstimate maintDate, treatmentType, notes, conservatorEmail)

artId, recordNumber -> costEstimate, maintDate, treatmentType, notes, conservatorEmail

---

Loan(artId, museumName, museumAddress, ownerEmail)

None

6 Normalization

Art(artId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType, artistId, location)

artId -> artistId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType

museumAddress -> location

artId+=artId, artistId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType

museumAddress+=museumAddress, location

Art is not in 3NF because museumAddress is not a superkey and location is not part of a key

Lossless join dependency preserving decomposition into 3NF is as follows

A minimum cover is

artId -> artistId, museumName, museumAddress, ownerEmail, creationYear, conditionStatus, dimensions, title, materialType

museumAddress -> location

Decompose on museumAddress -> location

R1(**museumAddress**, location)

R2(artId, **museumName, museumAddress, ownerEmail, materialType, artistId**, creationYear, conditionStatus, dimensions, title)

Every dependency is preserved

Lossless join dependency preserving decomposition into 3NF is

R1(**museumAddress**, location)

R2(artId, **museumName, museumAddress, ownerEmail, materialType, artistId**, creationYear, conditionStatus, dimensions, title)

Museum(museumName, museumAddress, city, foundingYear, annualVisitors, country)

museumName, museumAddress -> city, foundingYear, annualVisitors, country

museumAddress -> city, country

museumName, museumAddress+= museumName, museumAddress, city, foundingYear, annualVisitors, country

museumAddress+=museumAddress, city, country

Museum is not in 3NF because museumAddress is not a superkey and city, country is not a part of a key

Lossless join dependency preserving decomposition into 3NF is as follows

A minimum cover is

Standard form

museumName, museumAddress -> city

museumName, museumAddress -> foundingYear

museumName, museumAddress -> annualVisitors

museumName, museumAddress -> country

museumAddress -> city

museumAddress -> country

Minimize LHS

museumName, museumAddress -> foundingYear

museumName, museumAddress -> annualVisitors

museumAddress -> city

museumAddress -> country

Remove redundancy

museumName, museumAddress -> foundingYear

museumName, museumAddress -> annualVisitors

museumAddress -> city

museumAddress -> country

Decompose on museumAddress -> city

R1(museumAddress, city)

R2(museumName,**museumAddress**, foundingYear, annualVisitors, country)

Decompose on museumAddress -> country

R3(**museumAddress**, country)

R4(museumName,**museumAddress**, foundingYear, annualVisitors)

Every dependency is preserved

Lossless join dependency preserving decomposition into 3NF is

R1(museumAddress, city)

R3(**museumAddress**, country)

R4(museumName,**museumAddress**, foundingYear, annualVisitors)

Conservator(conservatorEmail, museumName, museumAddress, conservatorName, phoneNumber, specialization, certification)

conservatorEmail -> museumName, museumAddress, conservatorName, phoneNumber, specialization, certification

certification -> specialization

conservatorEmail+=conservatorEmail, museumName, museumAddress, conservatorName, phoneNumber, specialization, certification

certification+=certification, specialization

Conservator is not in 3NF because certification is not a superkey and specializationis not a part of a key

Lossless join dependency preserving decomposition into 3NF

A minimum cover is

conservatorEmail -> museumName, museumAddress, conservatorName, phoneNumber, specialization, certification

certification -> specialization

Decompose on certification -> specialization

R1(<u>certification</u>, specialization)

R2(<u>conservatorEmail</u>, **museumName, museumAddress**, conservatorName, phoneNumber, **certification**)

Every dependency is preserved

Lossless join dependency preserving decomposition into 3NF is

R1(<u>certification</u>, specialization)

R2(<u>conservatorEmail</u>, **museumName, museumAddress**, conservatorName, phoneNumber, **certification**)

Sculpture(artId, installationRequirement, weight)

artId -> installationRequirement, weight

artId+=artId, installationRequirement, weight

Sculpture is in 3NF because artId is a superkey

Painting(artId, surfaceType, paintType])

artId -> surfaceType, paintType

artId+=artId, surfaceType, paintType

Painting is in 3NF because artId is a superkey

Owner(ownerEmail, ownerName, ownerCountry, ownerType)

ownerEmail -> ownerName, ownerCountry, ownerType

ownerEmail+=ownerEmail, ownerName, ownerCountry, ownerType

Owner is in 3NF because ownerEmail is a superkey

Artist(artistId, artistName, birthYear, deathYear, nationality)

artistId -> artistName, birthYear, deathYear, nationality

artistId+=artistId, artistName, birthYear, deathYear, nationality

Artist is in 3NF because artistId is a superkey

Medium(materialType, storageRequirements, hazardLevel, durability)

materialType -> storageRequirements, hazardLevel, durability

hazardLevel -> durability

durability -> storageRequirements

hazardLevel+=hazardLevel, durability

Medium is not in 3NF because hazardLevel is not a superkey and durability is not a part of a key

Lossless join dependency preserving decomposition into 3NF

A minimum cover is

materialType -> storageRequirements, hazardLevel, durability

hazardLevel -> durability

durability -> storageRequirements

Decompose on durability -> storageRequirements

R1(durability, storageRequirements)

R2(materialType, hazardLevel, **durability**)

Decompose on hazardLevel -> durability

R3(hazardLevel, durability)

R4(materialType, **hazardLevel**)

Every dependency is preserved

Lossless join dependency preserving decomposition into 3NF is

R1(durability, storageRequirements)

R3(hazardLevel, durability)

R4(materialType, **hazardLevel**)

Guard(guardBadgeNumber, museumName, museumAddress, securityLevel, providerCompany, name, cost)

guardBadgeNumber -> museumName, museumAddress, securityLevel, providerCompany, name, cost

securityLevel, providerCompany -> cost

securityLevel, providerCompany+=securityLevel, providerCompany, cost

Guard is not in 3NF because securityLevel, providerCompany is not a superkey and cost is not a part of a key

Lossless join dependency preserving decomposition into 3NF

A minimum cover is

guardBadgeNumber -> museumName, museumAddress, securityLevel, providerCompany, name, cost

securityLevel, providerCompany -> cost

Decompose on securityLevel, providerCompany -> cost

R1(securityLevel, providerCompany, cost)

R2(guardBadgeNumber, **museumName, museumAddress**, securityLevel, **providerCompany**, name)

Every dependency is preserved

Lossless join dependency preserving decomposition into 3NF is

R1(securityLevel, providerCompany, cost)

R2(guardBadgeNumber, **museumName, museumAddress**, securityLevel, **providerCompany**, name)

MaintenanceRecord(artId, recordNumber, costEstimate, maintDate, treatmentType, notes, conservatorEmail)

artId, recordNumber -> costEstimate, maintDate, treatmentType, notes, conservatorEmail

artId, recordNumber+=artId, recordNumber, costEstimate, maintDate, treatmentType, notes, conservatorEmail

MaintenanceRecord is in 3NF because artId, recordNumber is a superkey

7 SQL DDL

```
CREATE TABLE Art1 (
   museumAddress  CHAR(20),
   location      CHAR(20),
   PRIMARY KEY (museumAddress),
   FOREIGN KEY (museumAddress) REFERENCES
   Museum1(museumAddress)
ON DELETE CASCADE
ON UPDATE NO ACTION
);
```

Justification for foreign keys: This table only holds the location and museum address for a given piece of art. When we delete a museum, any corresponding tuple in this table should be deleted automatically because the museum does not exist in the database (ON DELETE CASCADE). We would not expect the museum address for a given museum to change so we will throw an error if a user tries to change the address without modifying its children (ON UPDATE NO ACTION).

```
CREATE TABLE Art2 (
   artId          INT,
   museumName   CHAR(20),
   museumAddress  CHAR(20),
   ownerEmail     CHAR(20) NOT NULL,
   materialType     CHAR(20) NOT NULL,
   artistId         INT NOT NULL,
   creationYear     INT,
   conditionStatus CHAR(20),
   dimensions     CHAR(20),
   title         CHAR(20) NOT NULL,
   PRIMARY KEY (artId),

   FOREIGN KEY (museumName, museumAddress) REFERENCES
   Museum3(museumName, museumAddress)
ON DELETE SET NULL,
ON UPDATE NO ACTION,

   FOREIGN KEY (ownerEmail) REFERENCES
   Owner
ON DELETE NO ACTION
ON UPDATE CASCADE ,

   FOREIGN KEY (materialType) REFERENCES
   MediumMaterialHazard(materialType)
ON DELETE NO ACTION
ON UPDATE NO ACTION,

   FOREIGN KEY (artistId) REFERENCES
   Artist(artistId)
ON DELETE NO ACTION
ON UPDATE NO ACTION
);
```

Justification for foreign keys:

When a museum is deleted from the database, the art it had been housing should not be removed. We use ON DELETE SET NULL for museumName and museumAddress because a piece of art can exist without a museum. Because we do not expect the museum address or name to change we use ON UPDATE NO ACTION.

Every art must have an owner in our database, so we use ON DELETE NO ACTION for ownerEmail. However , we expect ownerEmail may change therefore we use ON UPDATE CASCADE so these changes will automatically populate the daughter tables.

Every art must have a medium and an artist in our database, therefore for the foreign keys artistId and materialType we use ON DELETE NO ACTION to prevent accidentally violating this. Also, materialType and artistId should always stay the same to we use ON UPDATA NO ACTION.

```
CREATE TABLE Painting (
artId         INT,
surfaceType     CHAR(20),
paintType     CHAR(20),
PRIMARY KEY (artId),
FOREIGN KEY (artId) REFERENCES Art2(artId)
     ON DELETE CASCADE
     ON UPDATE NO ACTION
);
```

Justification for foreign keys:
Because painting is a weak entity of art, deleting the corresponding artwork tuple should delete the painting tuple, hence ON DELETE CASCADE. artId should never change so we used ON UPDATE NO ACTION.

```
CREATE TABLE Sculpture (
artId             INT,
installationRequirement CHAR(20),
weight             INT,
PRIMARY KEY (artId),
    FOREIGN KEY (artId) REFERENCES Art2(artId)
ON DELETE CASCADE
ON UPDATE NO ACTION
);
```

Justification for foreign keys:
Like painting, sculpture is a weak entity of art, deleting the corresponding artwork tuple should delete the painting tuple, hence ON DELETE CASCADE. artId should never change so we used ON UPDATE NO ACTION.

```
CREATE TABLE MediumDurabilityPolicy (
durability         CHAR(20),
storageRequirements  CHAR(20),
PRIMARY KEY (durability)
);

CREATE TABLE MediumHazardDurability (
hazardLevel CHAR(20),
durability CHAR(20),
PRIMARY KEY (hazardLevel),
FOREIGN KEY (durability) REFERENCES MediumDurabilityPolicy(durability)
ON DELETE NO ACTION
ON UPDATE NO ACTION
);
```

Justification for foreign keys:
A medium entity should include both durability and hazard level attributes. We expect that once a medium is added into the database it will not be deleted because there can be more art work added to the database in the future and it could be made from the medium. We also expect that a 'mediums attributes will never change. Both foreign keys for the medium entity tables (durability in MediumHazardDurability and hazardLevel in MediumMaterialHazard) are set to ON

```
    DELETE NO ACTION and ON UPDATE NO ACTION because any medium entities should be complete and
    permanent.

CREATE TABLE MediumMaterialHazard (
materialType CHAR(20),
hazardLevel CHAR(20),
PRIMARY KEY (materialType),

FOREIGN KEY (hazardLevel) REFERENCES MediumHazardDurability(hazardLevel)
ON DELETE NO ACTION
ON UPDATE NO ACTION
);

CREATE TABLE MaintenanceRecord (
artId          INT,
recordNumber     INT,
costEstimate     DECIMAL(10,2) NOT NULL,
maintDate      DATE NOT NULL,
treatmentType     CHAR(20),
Notes          CHAR(100),
conservatorEmail CHAR(20),

PRIMARY KEY (artId, recordNumber),

FOREIGN KEY (artId) REFERENCES Art2(artId)
ON DELETE CASCADE
ON UPDATE NO ACTION,

FOREIGN KEY (conservatorEmail) REFERENCES Conservator1(conservatorEmail)
ON DELETE CASCADE
ON UPDATE CASCADE
);

Justification for foreign keys:
A maintenance record cannot exist without its corresponding conservator and art, so when an art or
    conservator is removed from the database their maintenance records disappear as well. artId
    should never change, but a conservatorEmail might. Therefore for both we set ON DELETE CASCADE,
     and for artId NO ACTION on update while conservator email has CASCADE on update.

CREATE TABLE Guard1 (
   securityLevel     INT,
   providerCompany CHAR(20) NOT NULL,
   cost          INT,

   PRIMARY KEY (securityLevel, providerCompany),

);



CREATE TABLE Guard2 (
   guardBadgeNumber INT,
   museumName     CHAR(20) NOT NULL,
   museumAddress  CHAR(20) NOT NULL,
   securityLevel     INT,
   providerCompany CHAR(20) NOT NULL,
   name          CHAR(20) NOT NULL,

   PRIMARY KEY (guardBadgeNumber)
```

```
   FOREIGN KEY (museumName, museumAddress) REFERENCES Museum3(museumName, museumAddress)
   ON DELETE CASCADE
   ON UPDATE NO ACTION

   FOREIGN KEY (securityLevel, providerCompany) REFERENCES Guard1(securityLevel, providerCompany)
ON DELETE CASCADE
ON UPDATE NO ACTION

);
```

Justification for foreign keys:
Every guard requires a museum, so when we delete a museum from the database any guard tuples that use that museumName and museumAddress should also be deleted. As mentioned previously museumName and museumAddress should never change. When a security level and providerCompany combination is deleted, all of the corresponding guard tuples should be deleted (we assume guards do not work independently). We use ON UPDATE NO ACTION for all foreign keys in this table because they should not be modified.

```
CREATE TABLE Owner (
   ownerEmail     CHAR(20),
   ownerName     CHAR(20) NOT NULL,
   ownerCountry     CHAR(20),
   ownerType     CHAR(20),

   PRIMARY KEY (ownerEmail)
);

CREATE TABLE Conservator1 (
   conservatorEmail CHAR(20),
   conservatorName CHAR(20) NOT NULL,
   phoneNumber     CHAR(20),
   certification     CHAR(20),
museumName     CHAR(20) NOT NULL,
museumAddress  CHAR(20) NOT NULL,

PRIMARY KEY (conservatorEmail)

FOREIGN KEY (certification) REFERENCES Conservator2(certification)
   ON DELETE NO ACTION
      ON UPDATE NO ACTION,

FOREIGN KEY (museumName, museumAddress) REFERENCES Museum3(museumName, museumAddress)
   ON DELETE CASCADE
      ON UPDATE NO ACTION,

);
```

Justification for foreign keys:
The certifications in Conservator2 should remain in the database even if no conservator uses them therefore we used ON DELETE NO ACTION. Also, the certifications should not change therefore we set ON UPDATE NO ACTION.

A conservator cannot exist without a museum, so when a museum is removed from the database, any conservator tuples should be removed as well. As mentioned previously museumName and museumAddress should not change. So like the other museum foreign keys we set ON DELETE CASCADE and ON UPDATE NO ACTION.

```
CREATE TABLE Conservator2 (
   certification     CHAR(20),
   specialization     CHAR(20),

   PRIMARY KEY (certification)
);




CREATE TABLE Museum1 (
   museumAddress  CHAR(20),
   city          CHAR(20) NOT NULL,

PRIMARY KEY (museumAddress)
);

CREATE TABLE Museum2 (
   museumAddress  CHAR(20),
   country      CHAR(20),

PRIMARY KEY (museumAddress)

FOREIGN KEY (museumAddress) REFERENCES Museum1(museumAddress)
   ON DELETE CASCADE
       ON UPDATE NO ACTION
);

Justification for foreign keys in Museum2 and Museum3:
If there is a tuple for a museumAddress and city in Museum1, there should also be a tuple for that
    museumAddress in Museum2 and Museum3. We use ON DELETE CASCADE to ensure that when you remove a
     museum from the database its information is removed from all tables. Because the address
    should not change we set ON UPDATE NO ACTION.

CREATE TABLE Museum3 (
   museumName     CHAR(20),
   museumAddress  CHAR(20),
   foundingYear     INT,
   annualVisitors     INT,

PRIMARY KEY (museumName, museumAddress)

FOREIGN KEY (museumAddress) REFERENCES Museum1(museumAddress)
   ON DELETE CASCADE
       ON UPDATE NO ACTION
);

CREATE TABLE Artist (
   artistId         CHAR(20),
   artistName     CHAR(20) NOT NULL,
   birthYear      INT,
   deathYear      INT,
   Nationality     CHAR(20),

   PRIMARY KEY (artistId)
);
CREATE TABLE Loan (
   artId          INT,
   museumName     CHAR(20),
```

```
   museumAddress  CHAR(20),
   ownerEmail     CHAR(20),

   PRIMARY KEY (artId, museumName, museumAddress, ownerEmail),

   FOREIGN KEY (artId) REFERENCES Art2(artId)
      ON DELETE CASCADE
      ON UPDATE NO ACTION,

   FOREIGN KEY (museumName, museumAddress) REFERENCES Museum3(museumName, museumAddress)
`       ON DELETE CASCADE
      ON UPDATE NO ACTION,

   FOREIGN KEY (ownerEmail) REFERENCES Owner(ownerEmail)
      ON DELETE CASCADE
      ON UPDATE CASCADE
);
```

Justification for foreign keys:
A loan cannot exist with out all three entities (Art, Owner, and Museum). Deleting any one of these should remove any corresponding loans from the database. Therefore we set ON DELETE CASCADE for artId, museumName, museumAddress, and ownerEmail. ownerEmail may change but artId and the museumName and museumAddress should not. Therefore we set ON UPDATE CASCADE for ownerEmail and ON UPDATE NO ACTION for artId, museumName, museumAddress.

8 Acknowledgement

No member of our team used any AI tools to complete this milestone.